CIS*3490 – Analysis & Design of Computer Algorithms - W26
Assignment 2

John Akinyemi & Gurjit Randhawa

Due: 13 February 2026, 11.59 pm

Instructions

Submit the assignment as a single PDF. All mathematical derivations must be shown step-by-step. For algorithmic questions, provide pseudo-code and a brief explanation of the complexity.

Use the Assignment #2 Dropbox to submit your assignment. The assignment must be submitted using **a single PDF** created using the LaTeX template provided that includes a listing of all the questions. **Late submissions will not be accepted**. The assignment must be **typed** and prepared using LaTeX (preferred) or another standard document editor. Failure to follow the required format will result in a significant penalty.

Total Marks: 30

Question 1 [5 marks]

Let

$$A = \langle a_1, a_2, \ldots, a_n \rangle$$

be an array of integers.

Design a **brute-force algorithm** that determines whether there exist indices

$$1 \leq i < j < k \leq n$$

such that

$$a_i + a_j = a_k.$$

Clearly state the **basic operation** and derive the **tight asymptotic time complexity** of your algorithm using Θ-notation.

For a brute force design, use 3 for-loops to check every possible combination of i, j, and k against the specifications.

```
for i <- to n-2:
  for j <- to n-1:
    for k <- to n:
       if A[i] + A[j] = A[k]:
        return true

return false
```

The basic operation is the comparison: A[i] + A[j] = A[k].
To find the tight asymptotic complexity, the algorithm can be written as a sum in term of its constant time operation.

To write this as a summation:

$\sum_{i=1}^{n-2} \sum_{j=i+1}^{n-1} \sum_{k=j+1}^{n} 1$

$= \sum_{i=1}^{n-2} \sum_{j=i+1}^{n-1} (n - (j+1) + 1)$

$= \sum_{i=1}^{n-2} \sum_{j=i+1}^{n-1} (n - j)$

When j = i+1, n-j = n-i-1
When j = i+2, n-j = n-i-2
When j = n-1, n-j = n-n+1 = 1

So, $\sum_{j=i+1}^{n-1}(n-j) = (n-i-1)+(n-i-2)+.....+1 = \frac{(n-i-1)(n-i)}{2}$ using arithmetic summation

We then have the following sum:

$\sum_{i=1}^{n-2} \frac{(n-i-1)(n-i)}{2}$

$= \frac{1}{2} \sum_{i=1}^{n-2} (n-i-1)(n-i)$

$= \frac{1}{2} \sum_{i=1}^{n-2} n^2 + i^2 - 2in - n + i$

Using rule $\sum(a+b) = \sum a + \sum b$

$= \frac{1}{2} (\sum_{i=1}^{n-2} n^2 - \sum_{i=1}^{n-2} 2in + \sum_{i=1}^{n-2} i^2 - \sum_{i=1}^{n-2} n + \sum_{i=1}^{n-2} i)$

$= \frac{1}{2} (n^2 \sum_{i=1}^{n-2} 1 - 2n \sum_{i=1}^{n-2} i + \sum_{i=1}^{n-2} i^2 - n \sum_{i=1}^{n-2} 1 + \sum_{i=1}^{n-2} i)$

$=\frac{1}{2}((n^2 \cdot (n-2)) - (2n \cdot \Theta(n^2)) + \Theta(n^3) - (n \cdot (n-2)) + (\Theta(n^2))$

$=\frac{1}{2}(\Theta(n^3) - \Theta(n^3) + \Theta(n^3) + \Theta(n^2) + (\Theta(n^2))$

Here, the dominant term is clearly $\Theta(n^3)$.
So, the tight asymptotic complexity is $\Theta(n^3)$.

Question 2 **[5 marks]**

A password consists of **exactly** n **characters**, where each character is either a digit

$$\{0, 1, \ldots, 9\}$$

or an uppercase letter

$$\{A, B, \ldots, Z\}.$$

An authentication system checks passwords using **exhaustive search**.

a. Express the **exact size** of the search space as a function of n.

b. Give a **tight lower bound** on the running time of any exhaustive-search algorithm that is guaranteed to find the correct password.

a) The set of digits contains 10 unique digits. The set of uppercase letters contains 26 letters. So, for each character of the passwords there is a possible 36 choices.
Thus, for a password of length n, there is 36^n possible combinations, i.e the search space is 36^n.

b) Since we seek a lower bound where it is **guaranteed** to find the correct password, we must assume the the worst case in which the correct password is found at the end of 36^n attempts (the last password that is tried). Since this means the full search space of 36^n combinations must be checked, the tight lower bound would be $\Omega(36^n)$.

Question 3 **[5 marks]**

Let $p_1 = (x_1, y_1)$ and $p_2 = (x_2, y_2)$ be two points in the plane. Define the **Manhattan distance** between p_1 and p_2 as

$$d_M(p_1, p_2) = |x_1 - x_2| + |y_1 - y_2|.$$

a. Prove that d_M satisfies the following properties for all points p_1, p_2, and p_3:

(i) $d_M(p_1, p_2) \geq 0$ for all p_1, p_2, and $d_M(p_1, p_2) = 0$ if and only if $p_1 = p_2$;

(ii) $d_M(p_1, p_2) = d_M(p_2, p_1)$;

(iii) $d_M(p_1, p_2) \leq d_M(p_1, p_3) + d_M(p_3, p_2)$.

b. Consider the brute-force closest-pair algorithm that computes the distance between every pair of points in a set of n points. If the Manhattan distance d_M is used instead of the Euclidean distance, does the *asymptotic running time* of the algorithm change in the worst case? Justify your answer.

a) (i) Given that $d_M(p_1, p_2) = |x_1 - x_2| + |y_1 - y_2|$ and $d_M(p_1, p_2) = 0$ iff $p_1 = p_2$,
If $p_1 = p_2$, then $x_1 = x_2$ and $y_1 = y_2$, so with $x_1 = x_2 = x$ and $y_1 = y_2 = y$:

$d_M(p_1, p_2) = |x - x| + |y - y| = |0| + |0| = 0$

Since $d_M(p_1, p_2) = 0$ when $p_1 = p_2$, and it is true for any absolute value of a real number n that $|n| \geq 0$, and the sum of any 2 non-negative integers is also non-negative, we can then say that $d_M(p_1, p_2) \geq 0$ for all p_1, p_2.

(ii) Given that $d_M(p_1, p_2) = |x_1 - x_2| + |y_1 - y_2|$, then $d_M(p_2, p_1) = |x_2 - x_1| + |y_2 - y_1|$
It is a basic property of absolute values that for an integer n, $|n| = |-n|$
Thus, $|x_1 - x_2| = |x_2 - x_1|$ and $|y_1 - y_2| = |y_2 - y_1|$
So, $|x_1 - x_2| + |y_1 - y_2| = |x_2 - x_1| + |y_2 - y_1|$ and $d_M(p_1, p_2) = d_M(p_2, p_1)$

(iii) Given $d_M(p_1, p_2) = |x_1 - x_2| + |y_1 - y_2|$ and $d_M(p_1, p_3) = |x_1 - x_3| + |y_1 - y_3|$ and $d_M(p_3, p_2) = |x_3 - x_2| + |y_3 - y_2|$

We can substitute those values into the given inequality:
$d_M(p_1, p_2) \leq d_M(p_1, p_3) + d_M(p_3, p_2)$
$= |x_1 - x_2| + |y_1 - y_2| \leq |x_1 - x_3| + |y_1 - y_3| + |x_3 - x_2| + |y_3 - y_2|$

For the x axis, we have: $|x_1 - x_2| \leq |x_1 - x_3| + |x_3 - x_2|$
Similarly, for the y axis: $|y_1 - y_2| \leq |y_1 - y_3| + |y_3 - y_2|$

We can prove the inequality using the triangle inequality: $|a + b| \leq |a| + |b|$.

For the x axis, let $a = x_1 - x_3$ and $b = x_3 - x_2$ and apply the triangle inequality

Then, $|x_1 - x_3 + x_3 - x_2| \leq |x_1 - x_3| + |x_3 - x_2|$
$= |x_1 - x_2| \leq |x_1 - x_3| + |x_3 - x_2|$①

For the y axis, let $a = y_1 - y_3$ and $b = y_3 - y_2$ and apply using the triangle inequality

Then, $|y_1 - y_3 + y_3 - y_2| \leq |y_1 - y_3| + |y_3 - y_2|$
$= |y_1 - y_2| \leq |y_1 - y_3| + |y_3 - y_2|$②

Add ① and ②

$|x_1 - x_2| + |y_1 - y_2| \leq |x_1 - x_3| + |x_3 - x_2| + |y_1 - y_3| + |y_3 - y_2|$

Therefore, $d_M(p_1, p_2) \leq d_M(p_1, p_3) + d_M(p_3, p_2)$

b) No, the asymptotic running time will not change in the worst case.
A brute force algorithm for the Manhattan distance will compare each set of pairs, and then compute the distance. The operations to compute the distance (subtraction, addition, absolute value) are all constant time operations. Represented as a summation, this will be:

$C(n) = \sum_{i=1}^{n-1} \sum_{j=i+1}^{1} 1 = \sum_{i=1}^{n-1} (n - i) = n \cdot (n - 1) = \Theta(n^2)$

The brute force closest pair algorithm for Euclidean distance is also $\Theta(n^2)$, and so they have the same asymptotic running time.

Question 4 [5 marks]

Consider an algorithm whose running time satisfies the recurrence:

$$T(n) = T(n-2) + cn \quad \text{for } n > 2, \qquad T(2) = d,$$

where c and d are positive constants.

a. Identify the **decrease-and-conquer variant** used by this algorithm.

b. Solve the recurrence and give a **tight asymptotic bound** for $T(n)$.

a) The variant used is decrease by constant (the constant is 2).

b) We are given $T(n) = T(n-2) + cn$ and $T(2) = d$

$i = 1$
$T(n) = T(n-2) + cn$①
$T(n-2) = T(n-4) + c(n-2)$②

Sub ② in ①.

$i = 2$
$T(n) = T(n-4) + c(n-2) + cn$③
$T(n-4) = T(n-6) + c(n-4)$④

Sub ④ in ③.

$i = 3$
$T(n) = T(n-6) + c(n-4) + c(n-2) + cn$⑤

Simplify ⑤:
$T(n) = T(n-6) + 3cn - 6c$

The pattern can be seen: $T(n) = T(n-2i) + icn + 2ic$
We have base case $T(2) = d$

$n - 2i = 2$
$2i = n - 2$
$i = \frac{n-2}{2}$

So, $T(n) = T(2) + \frac{n-2}{2}cn - 2\frac{n-2}{2}c$
$T(n) = d + c(n-2)(\frac{n}{2} - \frac{2}{2})$
$T(n) = d + c(n-2)(\frac{n}{2} - 1)$

Therefore, the tight asymptotic bound is $\Theta(n^2)$

Question 5 **[5 marks]**

Consider the following **ternary search** algorithm for searching a key K in a sorted array $A[0 \dots n-1]$.

If $n = 1$, compare K with the single element of the array. Otherwise, recursively compare K with $A[\lfloor n/3 \rfloor]$ and $A[\lfloor 2n/3 \rfloor]$ to determine in which third of the array the search should continue. This algorithm follows a **decrease-by-a-constant-factor strategy**, where the problem size is reduced by a factor of 3 at each step.

a. Set up a recurrence relation for the **worst-case number of key comparisons** performed by this algorithm. You may assume that $n = 3^k$ for some integer $k \geq 0$.

b. Solve the recurrence to obtain a **tight asymptotic bound** on the worst-case number of key comparisons.

c. Given that binary search performs approximately $\log_2 n + 1$ key comparisons in the worst case, compare the efficiency of ternary search with binary search in the worst case.

a) We are given the base case $n = 1$, where one comparison to the array's single element occurs.

Let $C(n)$ be the algorithm.
If $n > 1$, $C(n)$ will compare the key K with $A[\lfloor n/3 \rfloor]$ and $A[\lfloor 2n/3 \rfloor]$
So, in the worst case, $K \neq A[\lfloor n/3 \rfloor]$ and $K \neq A[\lfloor 2n/3 \rfloor]$
This is 2 comparisons, and since both evaluate to false, it will recursively investigate one third of the array.

So, we get the recurrence relation:
$C(n) = C(n/3) + 2$ for $n > 1$, and $C(1) = 1$

b) Assume $n = 3^k$ to solve.
$C(3^k) = C(\frac{3^k}{3}) + 2$, which can be written as $C(3^k) = C(3^{k-1}) + 2$

$i = 1$
$C(3^k) = C(3^{k-1}) + 2①$
$C(3^{k-1}) = C(3^{k-2}) + 2②$

Sub ② in ①

$i = 2$
$C(3^k) = C(3^{k-2}) + 4③$
$C(3^{k-2}) = C(3^{k-3}) + 2④$

Sub ④ in ③
$C(3^k) = C(3^{k-3}) + 6⑤$

A pattern emerges: $C(3^k) = C(3^{k-i}) + 2i$
Given base case $C(3^0) = 1$, assume $k - i = 0$, so $k = i$,

So, $C(3^k) = C(3^{k-k}) + 2k$
$C(3^k) = C(3^0) + 2k$
$C(3^k) = 1 + 2k$

Because we assume $n = 3^k$, $\log_3 n = \log_3(3^k)$ (take log of both sides) ①
Using the rule $\log_b(b^x) = x$ we get $\log_3(3^k) = k②$
Sub ② in ①
$\log_3 n = k$

We can use $\log_3 n = k$ in the recurrence relation. If $\log_3 n = k$, then
$C(3^k) = 1 + 2\log_3 n$

So, discarding constants, the tight asymptotic bound is $\Theta(\log_n)$

c) To compare ternary and binary search's worst case runtimes, convert $1 + 2\log_3 n$ to base 2.
Using change of base rule:

$2\log_3 n = 2 \cdot \frac{\log_2 n}{\log_2 3} = \ 1.26 \log_2 n$

So, binary search's worst case runtime is more efficient than that of ternary search.

Question 6 **[5 marks]**

Suppose two algorithms solve the same problem with running times:

$$T_1(n) = n^2 + 10n \quad \text{and} \quad T_2(n) = 5n \log_2 n.$$

a. Determine which algorithm is asymptotically more efficient.

b. Find an approximate value of n for which $T_2(n) < T_1(n)$.

c. Briefly explain why asymptotic analysis alone may be misleading for small input sizes.

a) $\lim_{n\to\infty} \frac{T(n)}{G(n)} = \lim_{n\to\infty} \frac{5n \log_2 n}{n^2+10n}$ to compare growth order

$\lim_{n\to\infty} \frac{5n \log_2 n}{n^2+10n}$
$= \lim_{n\to\infty} \frac{5n \log_2 n}{n(n+10)}$
$= \lim_{n\to\infty} \frac{5 \log_2 n}{n+10}$
$= \lim_{n\to\infty} \frac{\log_2 n}{n}$ simplified and removed constants

Using l'Hôpital's rule:
$= \lim_{n\to\infty} \frac{\frac{1}{nln2}}{1}$
$= 0$

This implies that $T_2(n) = 5n \log_2 n$ has a smaller growth rate than $T_1(n) = n^2 + 10n$ i.e is asymptotically more efficient.

b) We want to find a value of n such that $T_2(n) < T_1(n)$ (Using powers of 2 for ease of logarithm calculations).

When n = 1: $T_1(1) = 11, T_2(1) = 0, \rightarrow T_2(n) < T_1(n)$
When n = 2: $T_1(2) = 24, T_2(2) = 10, \rightarrow T_2(n) < T_1(n)$
When n = 4: $T_1(4) = 56, T_2(1) = 20 \rightarrow T_2(n) < T_1(n)$
.
.
.
When n = 128 $T_1(128) = 17664, T_2(128) = 4480 \rightarrow T_2(n) < T_1(n)$

Therefore, we can see that for all n, $T_2(n) < T_1(n)$.

c) Asymptotic analysis can be misleading for small terms because asymptotic analysis doesn't consider constant factors and lower order terms (e.g $n^2 + 10n$ just becomes $\Theta(n^2)$) and the initial cost for setup. At smaller inputs this becomes more significant, because an algorithm with low setup costs but a worse overall asymptotic efficiency may outperform an algorithm with a better asymptotic efficiency but a higher initial cost. Thus, asymptotic analysis may be misleading for a smaller input size.